通訊 3A 桑瑋佑 0106C033

目標:測量教室的長和寬，並且顯示其路徑。

上禮拜(利用期中跳動次數的程式去做修改):
幫我修改這個程式:
1.檢測 Z 軸的平均加速度來計算總直線距離
2.全部顯示的東西只剩下測量 Z 軸移動的總距離以及步數(Step)
3.其移動的總距離名稱為(Length)
4. 假設一步的步長為 0.9 公尺

結果:

在先前的程式中，步數的增加是基於 Z 軸加速度的變化來計算的。然而，即使步數沒有增加，移動距離仍然可能會增加，這是因為移動距離的計算不僅取決於步數，還取決於 Z 軸加速度的持續變化。

如果步數沒有增加，但移動距離增加了，這可能是因為在某些情況下 Z 軸加速度仍然有變化，導致移動距離的增加。例如，在行走過程中，即使步數沒有增加，但是當您走得更快或更慢時，Z 軸的加速度可能會有所變化，這將導致移動距離的增加。

所以這個方法導致測量距離誤差更大，所以這星期使用其他方式。

這星期:

使用這個程式，建立一個平面座標系統，並使用手機的計步器感應器記錄每個方向上的移動距離。
通過大幅度搖晃手機，記錄每個方向上的移動距離，每搖一次增加 0.6 米。
添加一個平面座標模式，將起點設置為(0,0)原點，並實時顯示移動的軌跡。
第一個距離顯示為長(Length)，第二個距離顯示為寬(Width)
完成長和寬的測量後，顯示最終的圖形。
利用角加速度去偵測(Z 軸)是否有轉向，然後再偵測寬

詢問 AI 後統整:

冷卻時間的實現：在 accListener 的 onSensorChanged 方法中加入了

ignoreData 標誌，並且在步數增加後，啟動一個 0.75 秒的延遲來重置這個標誌。
新增按鈕：在 AccDistanceScreen 組件中新增了一個按鈕，當按下按鈕時，觸發 onButtonClick 回調，並將 calculatingWidth 設置為 true 以開始計算寬度。
長度與寬度的區分：通過 calculatingWidth 標誌來決定當加速度超過門檻值時是增加長度還是寬度。
新增 PathCanvas 組件：用於繪製路徑圖。在這個組件中，使用 Canvas 繪製一個灰色的長方形來表示屏幕的邊界，並根據測量的長和寬繪製一個藍色的長方形來表示移動的路徑。
調整 PathCanvas 的大小：根據測量的長和寬計算比例，並按比例繪製藍色長方形，使其與測量的數據匹配。
新增變量 showPath：用於控制是否顯示路徑圖。
新增按鈕：在 AccDistanceScreen 中新增一個按鈕，當按下按鈕時，觸發 onShowPathClick 回調，將 showPath 設置為 true。
控制顯示：使用 if (showPath) 控制路徑圖的顯示。
更新 MainActivity 生命周期方法：在 onResume()和 onPause()方法中分別註冊和取消註冊陀螺儀監聽器。
實現陀螺儀監聽器：定義一個陀螺儀監聽器（gyroscopeListener），以接收感應器數據並相應地更新軌跡視圖。
使用 Compose 編寫 UI：創建一個 Compose 函數（RotationDirectionScreen）使用 Jetpack Compose 顯示 UI。
在 Compose 中更新軌跡：將上下文傳遞給 TrajectoryView，在 Compose 函數內顯示它。
處理權限：確保你處理應用程式中訪問感應器所需的權限。
測試應用程式：在實體設備或模擬器上運行應用程式，測試陀螺儀功能和軌跡視圖。
Add a gyroscope sensor to detect when the device turns.
Modify the logic to switch between measuring length and width based on the detected turns.

程式碼:

```
package com.example.myapplication

import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
```

```
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.myapplication.ui.theme.MyApplicationTheme
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

class MainActivity : ComponentActivity() {
    private lateinit var sensorManager: SensorManager
    private lateinit var accelerometer: Sensor
    private lateinit var gyroscope: Sensor
    private var totalLength by mutableStateOf(0.0)
    private var totalWidth by mutableStateOf(0.0)
    private var stepCount by mutableStateOf(0)
    private val threshold = 12.0f// 設置門檻值
    private val gyroThreshold = 4 // 角加速度門檻值，調整為更靈敏
    private var ignoreData = false // 是否忽略後續數據
    private var ignoreGyroData = false // 是否忽略角加速度數據
    private var calculatingWidth by mutableStateOf(false) // 是否正在計算寬度
    private var showPath by mutableStateOf(false) // 是否顯示路徑圖

    private val accListener = object : SensorEventListener {
        override fun onSensorChanged(event: SensorEvent) {
            if (event.sensor == accelerometer && !ignoreData) {
                val zAcceleration = event.values[2] // 只取得 Z 軸加速度
                if (zAcceleration > threshold) { // 當加速度超過門檻值
                    stepCount++
                    if (calculatingWidth) {
                        totalWidth += 0.60 // 每跳增加 60cm
                    } else {
                        totalLength += 0.60 // 每跳增加 60cm
                    }
                    ignoreData = true // 設置忽略後續數據的標誌
                    CoroutineScope(Dispatchers.Main).launch {
                        delay(750) // 0.75 秒後重置忽略標誌
                        ignoreData = false
                    }
                }
            }
        }
```

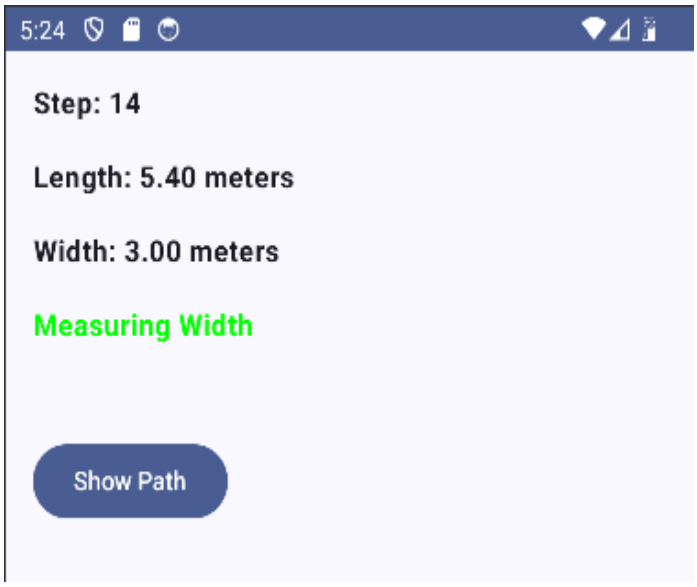
```
        override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
    }

    private val gyroListener = object : SensorEventListener {
        override fun onSensorChanged(event: SensorEvent) {
            if (event.sensor == gyroscope && !ignoreGyroData) {
                val zRotationRate = event.values[2] // 只取得 Z 軸角加速度
                if (kotlin.math.abs(zRotationRate) > gyroThreshold) {
                    calculatingWidth = !calculatingWidth
                    ignoreGyroData = true // 設置忽略後續數據的標誌
                    CoroutineScope(Dispatchers.Main).launch {
                        delay(900) // 0.9 秒後重置忽略標誌
                        ignoreGyroData = false
                    }
                }
            }
        }

        override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        sensorManager = getSystemService(SENSOR_SERVICE) as
SensorManager
        accelerometer =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)!!
        gyroscope =
sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE)!!
        setContent {
            MyApplicationTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    AccDistanceScreen(
                        totalLength = totalLength,
                        totalWidth = totalWidth,
                        stepCount = stepCount,
                        calculatingWidth = calculatingWidth,
                        onShowPathClick = {
                            showPath = true
                        },
                        showPath = showPath
                    )
                }
            }
        }
    }
```

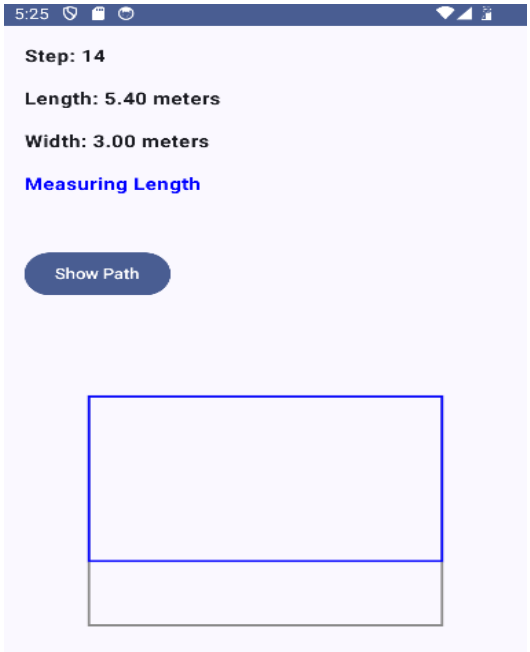
```
    override fun onPause() {
        super.onPause()
        sensorManager.unregisterListener(accListener)
        sensorManager.unregisterListener(gyroListener)
    }

    override fun onResume() {
        super.onResume()
        sensorManager.registerListener(
            accListener,
            accelerometer,
            SensorManager.SENSOR_DELAY_NORMAL
        )
        sensorManager.registerListener(
            gyroListener,
            gyroscope,
            SensorManager.SENSOR_DELAY_NORMAL
        )
    }
}

@Composable
fun AccDistanceScreen(
    modifier: Modifier = Modifier,
    totalLength: Double,
    totalWidth: Double,
    stepCount: Int,
    calculatingWidth: Boolean,
    onShowPathClick: () -> Unit,
    showPath: Boolean
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
            .verticalScroll(rememberScrollState()),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            text = "Step: $stepCount",
            fontWeight = FontWeight.Bold
        )
        Text(
            text = "Length: %.2f meters".format(totalLength),
            fontWeight = FontWeight.Bold
        )
        Text(
            text = "Width: %.2f meters".format(totalWidth),
            fontWeight = FontWeight.Bold
        )
```

```
        Text(
            text = if (calculatingWidth) "Measuring Width" else "Measuring
Length",
            color = if (calculatingWidth) Color.Green else Color.Blue,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = onShowPathClick) {
            Text("Show Path")
        }
        Spacer(modifier = Modifier.height(16.dp))
        if (showPath) {
            PathCanvas(totalLength = totalLength, totalWidth = totalWidth)
        }
    }
}

@Composable
fun PathCanvas(totalLength: Double, totalWidth: Double) {
    Canvas(modifier = Modifier
        .fillMaxWidth()
        .height(300.dp)
        .padding(16.dp)) {
        val rectWidth = size.width * 0.8f
        val rectHeight = size.height * 0.8f
        val left = (size.width - rectWidth) / 2
        val top = (size.height - rectHeight) / 2
        val right = left + rectWidth
        val bottom = top + rectHeight

        drawRect(
            color = Color.Gray,
            topLeft = androidx.compose.ui.geometry.Offset(left, top),
            size = androidx.compose.ui.geometry.Size(rectWidth, rectHeight),
            style = Stroke(width = 5f)
        )

        val scaleX = rectWidth / totalLength.toFloat()
        val scaleY = rectHeight / totalWidth.toFloat()
        val rectScale = minOf(scaleX, scaleY)

        val scaledLength = totalLength.toFloat() * rectScale
        val scaledWidth = totalWidth.toFloat() * rectScale

        drawRect(
            color = Color.Blue,
            topLeft = androidx.compose.ui.geometry.Offset(left, top),
            size = androidx.compose.ui.geometry.Size(scaledLength,
scaledWidth),
            style = Stroke(width = 5f)
```

```
        )
    }
}

@Preview(showBackground = true)
@Composable
fun DefaultPreview() {
    MyApplicationTheme {
        AccDistanceScreen(
            totalLength = 10.0,
            totalWidth = 5.0,
            stepCount = 15,
            calculatingWidth = false,
            onShowPathClick = {},
            showPath = false
        )
    }
}
```

結果圖:




5:25
Step: 14
Length: 5.40 meters
Width: 3.00 meters
Measuring Length
Show Path